

Programming as a way of thinking

COSC 1210 (CS I) FA26 | Augustana U.

Table of Contents

- [1. Objectives](#)
- [2. What is programming?](#)
- [3. Why learn programming?](#)
- [4. Why programming in Python?](#)
- [5. Glossary](#)
- [6. References](#)

1. Objectives

Find out:

- What is programming?
- Why learn programming?
- Why program in Python?

Along the way, you see:

- How to open and use a Jupyter notebook
- 32 different technical terms or commands
- How to define a function
- How to handle a program exception
- How to solve a computational problem
- How to generalize the solution of a computational problem
- How to visualize data

2. What is programming?

In your browser, open a [Jupyter notebook](#) to code along: go to <https://jupyter.org/try-jupyter/> (this is JupyterLite in the cloud).

Alternatively, you can download Jupyter lab from jupyter.org ("Install JupyterLab").

Code along exercises

- There is at least one code along exercise in every class that you need to upload.
- You can download the .ipynb file from Jupyter and upload it to Canvas.
- At the end of today's session, upload your .ipynb file to Canvas
- I'm going to switch to Jupyter myself now: jupyter.org/try-jupyter

Jupyter notebooks

- There is a short introduction (you don't have to read it).
- There is a launcher with different available apps (click "+")

- Open the "Python (Pyodide)" notebook
- Click on the title "Untitled.ipynb" and change it to "Demo.ipynb"
- A notebook consists of input and output cells
- You can choose between raw text, code, or markdown cells
- Change the first cell to markdown and enter a title: "Telling a machine what to do" - render it by entering SHIFT + ENTER
- Open the markdown cell again and add "### " (level 3 headline)
- It's simple: you code in the code cells, you write in the text cells, and you can add markdown code to change the layout
- In the sidebar, you have a file list - that's it!

Telling a machine what to do

For example:

```
def hello(name="there"):
    print(f"Hello, {name}.")

hello()
hello("Marcus")
hello("you nerd")
```

```
Hello, there.
Hello, Marcus.
Hello, you nerd.
```

Telling a machine what not to do

For example:

```
numerator = 1
denominator = 0
try:
    result = numerator / denominator
    print(result)
except ZeroDivisionError as e:
    print(f"Error: {e}")
```

```
Error: division by zero
```

Learning how to solve problems

For example:

When a party of four splits a bill of \$45.50 with a tip of 15% evenly, how much does every one have to pay?

```
bill = 45.50
tax = 15
party = 4
total = bill + (bill * tax/100)
share = total/party
```

```
print(share)
print(f"The per-person share is: ${share:.2f}")
```

```
13.08125
The per-person share is: $13.08
```

Learning how to generalize solutions

For example:

When a party splits its bill with a given percentage tip evenly, how much does every one have to pay?

```
raw = input("Enter bill, tax, party size:\n")
parts = raw.split(sep=',') # using str.split()
bill, tax, party = (float(p) for p in parts) # using a generator
total = bill + (bill * tax/100) # using arithmetic operators
share = total/party
print(f"The share of ${bill} for {int(party)} with {tax}% tip is ${share:.2f}")
```

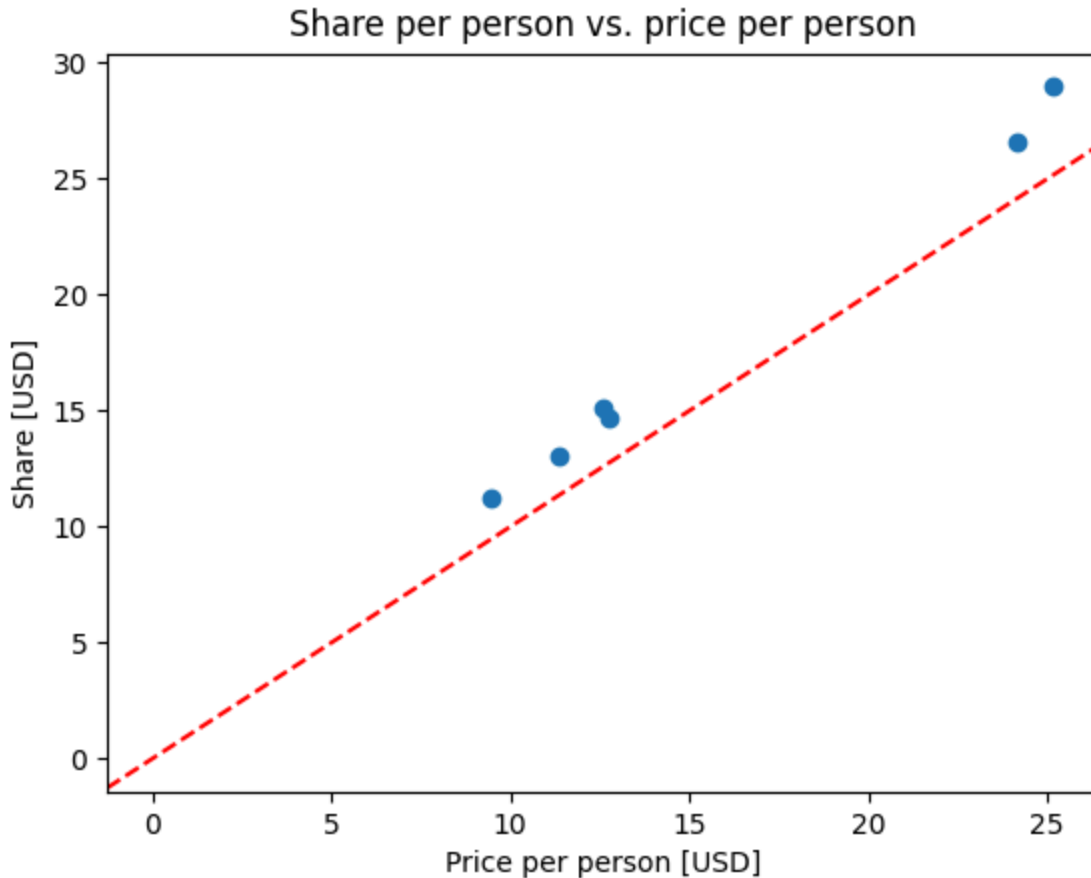
Learning how to visualize results

For example:

I ran the tax calculator on six different bills. How well does the price per person (bill/party) before tip predict what you end up paying (share)?

```
import matplotlib.pyplot as plt
import numpy as np

bill = np.array([45.50, 145.0, 25.18, 38.23, 100.75, 66.07])
share = np.array([13.08, 26.58, 15.11, 14.65, 28.97, 11.27])
party = np.array([4, 6, 2, 3, 4, 7])
plt.scatter(bill/party, share)
plt.axline((0,0), (1,1), color='red', linestyle='--') # tax = 0%
plt.xlabel("Price per person [USD]")
plt.ylabel("Share [USD]")
plt.title("Share per person vs. price per person")
```



Answer:

Your per-person share is almost entirely just your pre-tip cost per person, nudged up by a fixed 10-25% depending on that visit's tip.

Lessons

- It's never natural to ask a machine to do a job for you.
- There's too much stuff to remember so don't even try.
- Instead make ample use of available documentation.
- Learn a few principles and facts to get faster.

3. Why learn programming?

- A valid question especially in times of generative AI! (A much discussed question among experts, see [this IEEE article](#).)
- What do you think?
 1. Independence (self-sufficiency, freedom)
 2. Understand what AI does and control it
 3. Make tools to make your work easier

4. Learn how to use machines for problem-solving

- What do I think?

1. Programming builds empirical, iterative thinking: it's immediately falsifiable ("Did it work?")¹
2. Programming is a form of autonomy (freedom): you can turn your ideas into a working product without asking for permission, money, or someone else's tool.
3. Programming is a form of agency (power): it is satisfying in itself to build something that works from nothing.
4. Programming trains technological citizenship: hiring, credit, health, etc. are governed by IT and AI. Be the master of the machines, and not their slave.
5. You can't verify what you can't read: AI makes mistakes and cannot reason about your specific problem.
6. AI does not lower the bar, it raises it: getting something to run is easy now, knowing when and how to deploy agents is difficult.

- When and how did I start programming?

In 1979, when I was 16, my dad bought me a TI-99/4 home computer. My best friend had an Apple II Plus². We began to write computer games and eventually both ended up getting our PhDs in physics. It's fun to compare what Linus Torvalds, the creator Linux (and still its head honcho) [has to say about "The beauty of programming"](#).



- What if you're not interested in Computer Science or computers as such?

The further you get into your field of interest - be it archeology, sociology, mathematics, exercise science or nursing - the more you will depend on computing resources and computational thinking, simply because we ask more and more from our machines. And they don't think for themselves (and never will, IMHO).

4. Why programming in Python?

- Popularity (see [TIOBE](#) and [IEEE](#) indices of top programming languages).
 1. Python has ranked #1 (or close) on the TIOBE and IEEE Spectrum indices for years - the language most likely to still matter by the time you graduate.
 2. A larger user base means a larger ecosystem: more libraries, more tutorials, more people online who can help you get unstuck.
 3. It's the default language of data science and AI/ML - the field this course is a stepping stone toward.
- Simplicity

1. Python's syntax reads close to plain English: no semicolons, no type declarations, minimal boilerplate standing between you and your first working program.
2. That means class time goes to computational thinking, not fighting the compiler - the actual skill this course is trying to teach.
3. It was explicitly designed for readability, which matters most when it's you, three weeks from now, reading your own code back.

- Expressivity

1. Python lets you express complex ideas in very few lines - one list comprehension can replace a five-line loop; your six-line tax calculator would take much longer to write in many other languages.
2. Dynamic typing and a batteries-included standard library mean less code spent declaring types or importing basic functionality - more of your effort goes toward the problem itself, not the paperwork.
3. Being able to prototype an idea in minutes and immediately see the result is what makes predict-run-modify learning possible - you experiment instead of just reading about syntax.

- Some things also speak against Python as a first language:

1. Dynamic typing catches fewer bugs at the point you make them - a type error Java or C would flag immediately might not surface in Python until the program actually runs that line, sometimes weeks later.
2. It hides what's happening underneath: no manual memory management, no compilation step, so you don't build the intuition for how a computer actually executes a program - an intuition that matters once you reach a low-level language like C, or a systems/computer-architecture course.
3. Indentation-defined blocks are elegant on the page but a real source of invisible bugs: mixing tabs and spaces, or copy-pasting code at the wrong indent level, can break a program with no visible cue.
4. It's slow compared to compiled languages, which is invisible in a classroom exercise but becomes a real constraint the moment performance actually matters.

5. Glossary

These are the technical terms that were mentioned in this one lecture alone - too much to learn by heart. The only way to develop and keep your tech vocabulary is to use the tech - in this case: by coding.

Term	Definition
1 .ipynb	the file extension for a Jupyter notebook; stores each cell and its output as JSON.
2 arithmetic operators	symbols like +, -, *, / used to perform mathematical calculations on numbers.
3 code cell	a notebook cell that contains runnable code, as opposed to a Markdown cell of formatted text.
4 command mode	the notebook state where keyboard shortcuts

Term	Definition
	act on the cell itself rather than its text; entered by pressing Esc.
5 command-mode shortcuts	M converts the selected cell to Markdown, Y converts it to Code, A inserts a new cell above, B inserts a new cell below.
6 compiled language	a language whose source code is translated into machine code by a compiler before it runs, unlike an interpreted language such as Python.
7 computational thinking	breaking a problem into precise, ordered steps a machine can execute - the core skill this course teaches, independent of any one language.
8 default argument	a parameter's fallback value used when the caller doesn't supply one, e.g. name="there".
9 dynamic typing	a design where a variable's type is determined at runtime rather than declared up front, so the same name can hold different kinds of values over its life.
10 Esc	the key that exits edit mode and enters command mode in a notebook.
11 exception	an error that occurs while a program is running, which can be caught and handled instead of crashing.
12 f-string	a string literal prefixed with f that lets you embed expressions inside curly braces, e.g. {name}.
13 float()	a built-in function that converts a value (often a string) into a decimal number.
14 format specifier	inside an f-string, controls how a value is

Term	Definition
	displayed, e.g. <code>{share: .2f}</code> rounds to 2 decimal places.
15 function	a named, reusable block of code that performs a specific task, defined with <code>def</code> .
16 generator expression	a compact way to build a lazy, one-time iterator, written like a list comprehension but with <code>()</code> instead of <code>[]</code> .
17 import	a statement that loads code from another module so its functions can be used.
18 indentation	in Python, whitespace at the start of a line that defines which block a statement belongs to, instead of braces <code>{ }</code> like many other languages.
19 input()	a built-in function that pauses the program and reads a line of text typed by the user.
20 Jupyter	an open-source system for interactive notebooks mixing code, text, and output; the name blends Julia, Python, and R, its first three supported languages.
21 list	an ordered, changeable collection of values, e.g. <code>[13.08, 26.58, 15.11]</code> .
22 list comprehension	a compact way to build a list in one line, e.g. <code>[float(p) for p in parts]</code> , evaluated eagerly (unlike a generator expression).
23 memory management	allocating and freeing the computer's memory as a program runs; Python does this automatically, hiding it from the programmer.
24 module	a file of reusable Python code (e.g. <code>matplotlib.pyplot</code>) that can be imported into another program.
25 notebook	a document made of cells (code or Markdown)

Term	Definition
	that runs interactively, mixing code, output, and explanation.
26 <code>plt.scatter()</code>	a matplotlib function that draws a scatter plot from two lists of x- and y-values.
27 Pyodide	a distribution of Python compiled to WebAssembly, letting Python run directly in the browser - what JupyterLite uses instead of a remote server.
28 standard library	the collection of modules that ships with Python itself (e.g. <code>csv</code> , <code>random</code>), usable without installing anything extra.
29 <code>str.split()</code>	a string method that breaks a string into a list of substrings, using a separator.
30 <code>try/except</code>	a construct that lets a program attempt code that might fail and handle the failure gracefully instead of crashing.
31 tuple unpacking	assigning several variables at once from the values produced by an iterable, e.g. <code>~bill, tax, party = ...~</code> .
32 variable	a name that refers to a value stored in memory.

6. References

- Pandey, R. (2025, October 9). Learning to code still matters in the age of AI. *IEEE Spectrum*. ([Link](#))
- Torvalds, L. (2001). The beauty of programming. Bryn Mawr College Computer Science. ([Link](#))

Footnotes:

¹ Falsifiable vs. verifiable: 1) A claim is falsifiable if there is one possible observation that would prove it wrong: "All swans are white" is proven wrong if you can find one black swan. 2) A claim is verifiable if there is some possible observation that would prove it right: "There exists a black swan" is verifiable (find one), but not falsifiable (you can never finish checking that none exist anywhere). "All swans are white" is falsifiable but not verifiable: you cannot test an infinite number of swans.

² These home computers cost around \$1,000 in 1979, which is about \$5,000 in today's money. To enter computing then was not a poor kid's game. Today, you can start with a \$5 micro:bit computer and MicroPython, if you want the hardware, or just use your browser.

Author: Marcus Birkenkrahe

Created: 2026-09-07 Mon 20:45